

ESCOLINHA DO PROFESSOR GIRAFALES

ENGENHARIA DA COMPUTAÇÃO, 5º PERÍODO

Autores:

- João Paulo Santos Botelho
- Juan Sobral Costa

Afya UnifipMOC

Resumo:

O presente projeto consiste no desenvolvimento de um sistema escolar em Python, utilizando os conceitos de Programação Orientada a Objetos (POO) para gerenciar alunos, professores e responsáveis. O sistema implementa funcionalidades de login, cadastro, consulta e atualização de informações, aplicando os princípios de abstração, encapsulamento, herança e polimorfismo. Por meio de classes bem estruturadas, como Usuarios, Aluno, Professor, Pai, MateriaAluno e Escola, foi possível organizar dados em memória e criar um fluxo funcional de interação, incluindo a manipulação de notas e perfis de usuários. O projeto ainda contempla uma interface simples para visualização de dados, garantindo praticidade e segurança no gerenciamento das informações escolares.

Sumário

Introdução	3
Como foi feito?.....	3
1. Planejamento e definição do escopo	3
2. Modelagem de classes e diagrama UML	3
3. Implementação das classes em Python.....	4
1. Classe Usuarios:	4
2. Classe Aluno (herda de Usuarios)	5
3. Classe MateriaAluno.....	5
4. Classe Professor (herda de Usuarios).....	6
5. Classe Pai (herda de Usuarios)	6
6. Classe Escola	6
4. Desenvolvimento do módulo de autenticação	7
5. Criação da interface gráfica (GUI).....	7
6. Testes e validação	7
Resultados	8
Conclusão	11

Figuras:

Figura 1 - Desenho Técnico	4
Figura 2 - Login	8
Figura 3 - Area do Aluno	9
Figura 4 - Area do Professor	10

Introdução

O presente relatório descreve o desenvolvimento de um sistema de gestão escolar, concebido como parte de um projeto acadêmico para aplicação prática dos conceitos de Programação Orientada a Objetos (POO) em Python. O objetivo principal do sistema é simular a rotina de uma escola, permitindo o cadastro e gerenciamento de alunos, professores e responsáveis, bem como o registro e consulta de notas por disciplina.

O projeto foi estruturado utilizando os pilares fundamentais da POO: **abstração**, **encapsulamento**, **herança** e **polimorfismo**. A classe base `Usuarios` foi criada para gerenciar autenticação e controle de acesso, enquanto classes derivadas como `Aluno`, `Professor` e `Pai` representam entidades reais do contexto escolar, garantindo organização e modularidade no código. Além disso, o sistema utiliza coleções em memória, como listas, para armazenar e manipular objetos, proporcionando consultas rápidas e eficientes.

O sistema conta com uma interface gráfica simples, construída em `customtkinter`, que possibilita a interação do usuário de forma intuitiva, permitindo login, cadastro de alunos e pais, visualização de boletins e lançamento de notas pelos professores. O desenvolvimento incluiu a criação de um diagrama UML no `Draw.io`, que ilustra a estrutura de classes, atributos e relacionamentos, servindo como guia para a implementação.

O relatório detalha, a seguir, os objetivos do projeto, a metodologia utilizada, os resultados obtidos e as conclusões sobre a aplicação dos conceitos de POO no desenvolvimento de um sistema funcional e organizado. Espaços ao longo do documento estão reservados para a inclusão de imagens e diagramas, como o UML, que complementam a documentação técnica e visual do projeto.

Como foi feito?

O desenvolvimento do sistema escolar seguiu uma abordagem estruturada, dividida em etapas que permitiram a implementação gradual de funcionalidades, respeitando os princípios da Programação Orientada a Objetos (POO) em Python. As etapas e procedimentos adotados foram:

1. Planejamento e definição do escopo

- Escolha do tema: sistema escolar com cadastro e gerenciamento de alunos, professores e responsáveis.
- Definição das funcionalidades principais: login de usuários, cadastro de alunos e pais, lançamento de notas, visualização de boletins e listagem de entidades.
- Identificação das entidades e suas relações para construção do diagrama UML.

2. Modelagem de classes e diagrama UML

- Criação do diagrama de classes no `Draw.io`, representando a estrutura do sistema com todas as classes, atributos e métodos.
- Estabelecimento de relacionamentos entre classes, incluindo herança (`Usuario` como classe base, com `Aluno`, `Professor` e `Pai` como subclasses), encapsulamento de atributos sensíveis (como senha) e polimorfismo nos métodos de manipulação de dados.

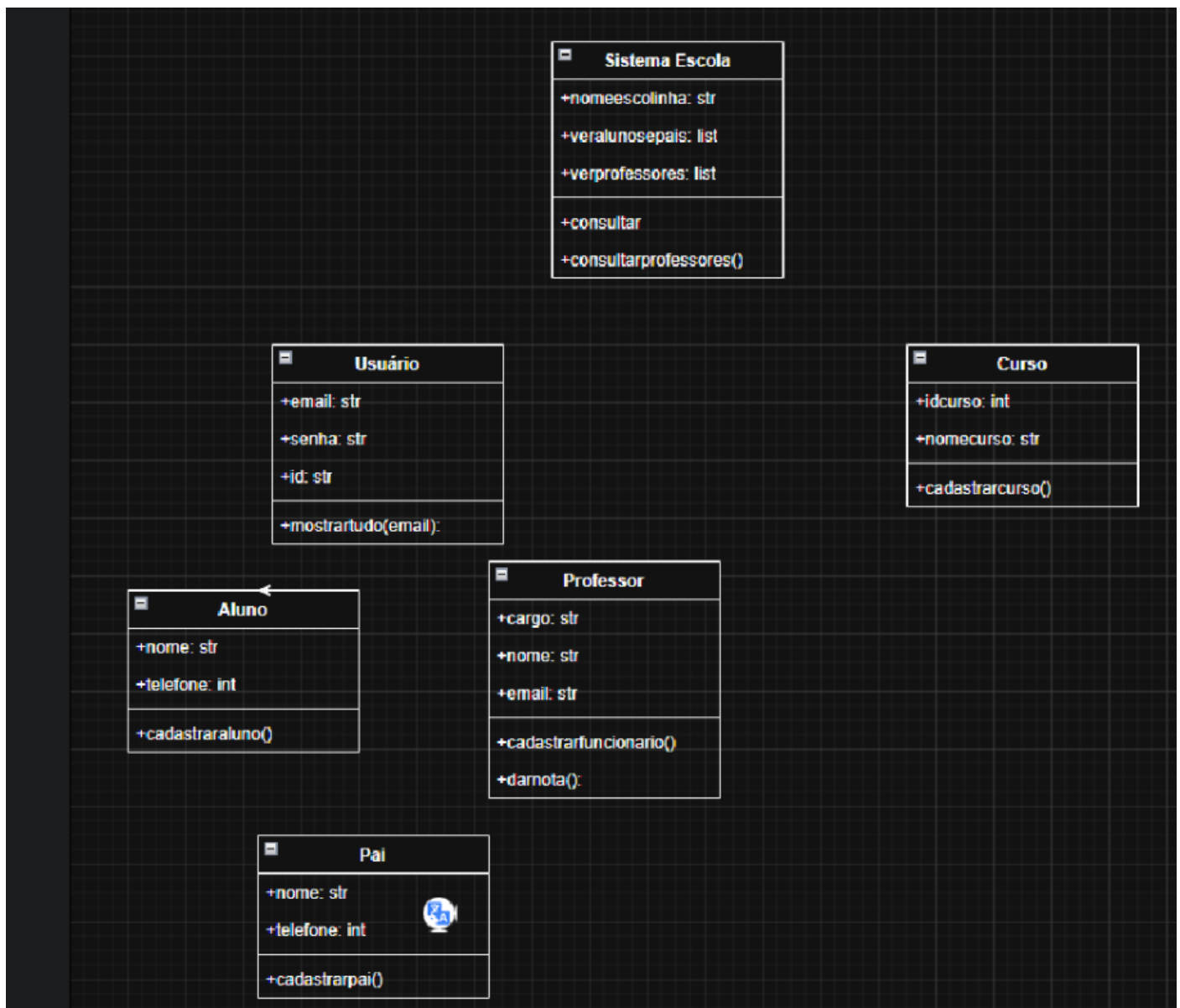


Figura 1 - Desenho Técnico

3. Implementação das classes em Python

No desenvolvimento do sistema, foram implementadas seis classes principais, cada uma representando uma entidade real do contexto escolar e aplicando os conceitos de Programação Orientada a Objetos, como abstração, encapsulamento, herança e polimorfismo. A seguir, detalhamos cada classe:

1. Classe Usuarios:

A classe base para todos os perfis do sistema, representando um usuário genérico, seja aluno, professor ou responsável (pai).

- **Atributos principais:**

- nome: Nome completo do usuário.
- cpf: CPF do usuário, utilizado como identificador único.
- `_senha`: Atributo privado que armazena a senha do usuário, protegido para evitar acesso direto.

- userid: Identificador único gerado automaticamente para cada usuário.
- **Encapsulamento e controle de acesso:**
 - O atributo `_senha` é privado e só pode ser alterado pelo método `set_senha`, garantindo validação.
 - A propriedade `senha` permite exibir a senha de forma protegida (*****) sem revelar seu valor real.
- **Métodos principais:**
 - `set_senha(senha)`: Define a senha, garantindo mínimo de 4 caracteres.
 - `verificar_senha(senha)`: Valida se a senha fornecida confere com a cadastrada.
- **Objetivo:** Servir como classe base para herança, permitindo reaproveitamento de atributos e métodos em classes filhas (Aluno, Professor, Pai) e aplicando polimorfismo na manipulação de diferentes tipos de usuário.

2. Classe Aluno (herda de Usuarios)

Representa os estudantes cadastrados no sistema.

- **Atributos adicionais:**
 - `nascimento`: Data de nascimento do aluno.
 - `email`: Email de contato.
 - `materias`: Lista de objetos `MateriaAluno`, cada um representando uma disciplina e sua nota.
 - `pai`: Lista de objetos `Pai` responsáveis pelo aluno.
- **Funcionalidade:**
 - Herda métodos de autenticação da classe `Usuarios`.
 - Permite armazenar notas por disciplina e relacionar o aluno a seus responsáveis.

3. Classe MateriaAluno

Representa cada disciplina cursada por um aluno, incluindo a nota.

- **Atributos:**
 - `nome`: Nome da disciplina (ex: Matemática, Física).
 - `nota`: Nota atual do aluno na disciplina.
- **Funcionalidade:**
 - Facilita o armazenamento estruturado das notas de cada aluno.
 - Permite que o professor atualize notas individualmente, garantindo organização dos dados.

4. Classe Professor (herda de Usuarios)

Representa os professores do sistema.

- **Atributos adicionais:**
 - nascimento: Data de nascimento do professor.
 - email: Email de contato.
 - materia: Disciplina que o professor leciona.
- **Funcionalidade:**
 - Permite autenticação e diferenciação de permissões dentro do sistema.
 - Facilita lançamento de notas e consulta à lista de alunos.

5. Classe Pai (herda de Usuarios)

Representa os responsáveis pelos alunos.

- **Atributos adicionais:**
 - nascimento: Data de nascimento do pai ou responsável.
 - email: Email de contato.
- **Funcionalidade:**
 - Permite criar contas vinculadas a alunos e gerenciar informações de filhos.
 - Garante que pais e alunos possam ter acessos distintos e seguros.

6. Classe Escola

Representa a instituição escolar e serve como controlador central do sistema.

- **Atributos principais:**
 - nomeescola: Nome da escola.
 - alunos: Lista de alunos cadastrados.
 - funcionarios: Lista de professores cadastrados.
 - materias: Lista de disciplinas disponíveis na escola.
- **Métodos principais:**
 - cadastrarcliente(aluno): Adiciona um novo aluno à lista da escola.
 - cadastrarfuncionario(funcionario): Adiciona um novo professor à lista de funcionários.
 - adicionarnota(aluno_id, materia_nome, nova_nota): Atualiza a nota de um aluno em uma disciplina específica.

- **Objetivo:**

- Centralizar o gerenciamento de usuários e disciplinas.
- Permitir operações de cadastro, consulta e atualização de notas de forma organizada e segura.

Essas classes, em conjunto, garantem a aplicação dos princípios de POO:

- Abstração: Cada classe representa uma entidade do sistema escolar.
- Encapsulamento: Senhas e dados sensíveis são protegidos com atributos privados e propriedades.
- Herança: Aluno, Professor e Pai herdam de Usuarios, reaproveitando código.
- Polimorfismo: Métodos como verificar_senha podem ser utilizados por diferentes tipos de usuário.
- Organização em memória: Listas de alunos, professores e matérias armazenam objetos de forma estruturada.

4. Desenvolvimento do módulo de autenticação

- Implementação de login com verificação de senha, garantindo que apenas usuários cadastrados possam acessar áreas restritas do sistema.
- Diferenciação de roles (Aluno ou Professor) para fornecer acesso a funcionalidades específicas de cada perfil.

5. Criação da interface gráfica (GUI)

- Utilização da biblioteca customtkinter para desenvolver telas de login, cadastro e áreas específicas para alunos e professores.
- Implementação de elementos interativos como botões, caixas de texto, menus e campos de listagem, permitindo interação intuitiva do usuário.
- Integração com o backend de classes Python, garantindo que as ações na interface alterem os objetos e coleções em memória.

6. Testes e validação

- Testes de cadastro, login, lançamento de notas e exibição de boletins para garantir funcionamento correto do sistema.
- Correção de inconsistências na manipulação de objetos e validação de entradas de dados, como verificação de notas numéricas e campos obrigatórios.

Essa metodologia permitiu organizar o desenvolvimento de forma iterativa, garantindo que cada componente fosse implementado, testado e validado antes de avançar para a etapa seguinte. O uso do diagrama UML serviu como guia visual e técnico para a implementação das classes e seus relacionamentos.

Resultados

O desenvolvimento do sistema escolar resultou em uma aplicação funcional que atende aos objetivos propostos, com foco em cadastro, gerenciamento e visualização de informações de alunos, pais e professores. Os principais resultados obtidos incluem:

1. Cadastro e gerenciamento de usuários

- Implementação de cadastro de alunos e responsáveis (pais), permitindo que cada pai crie a conta de seus filhos.
- Cadastro de professores com disciplina associada.
- Controle de autenticação via login, com diferenciação de papéis (Aluno e Professor).

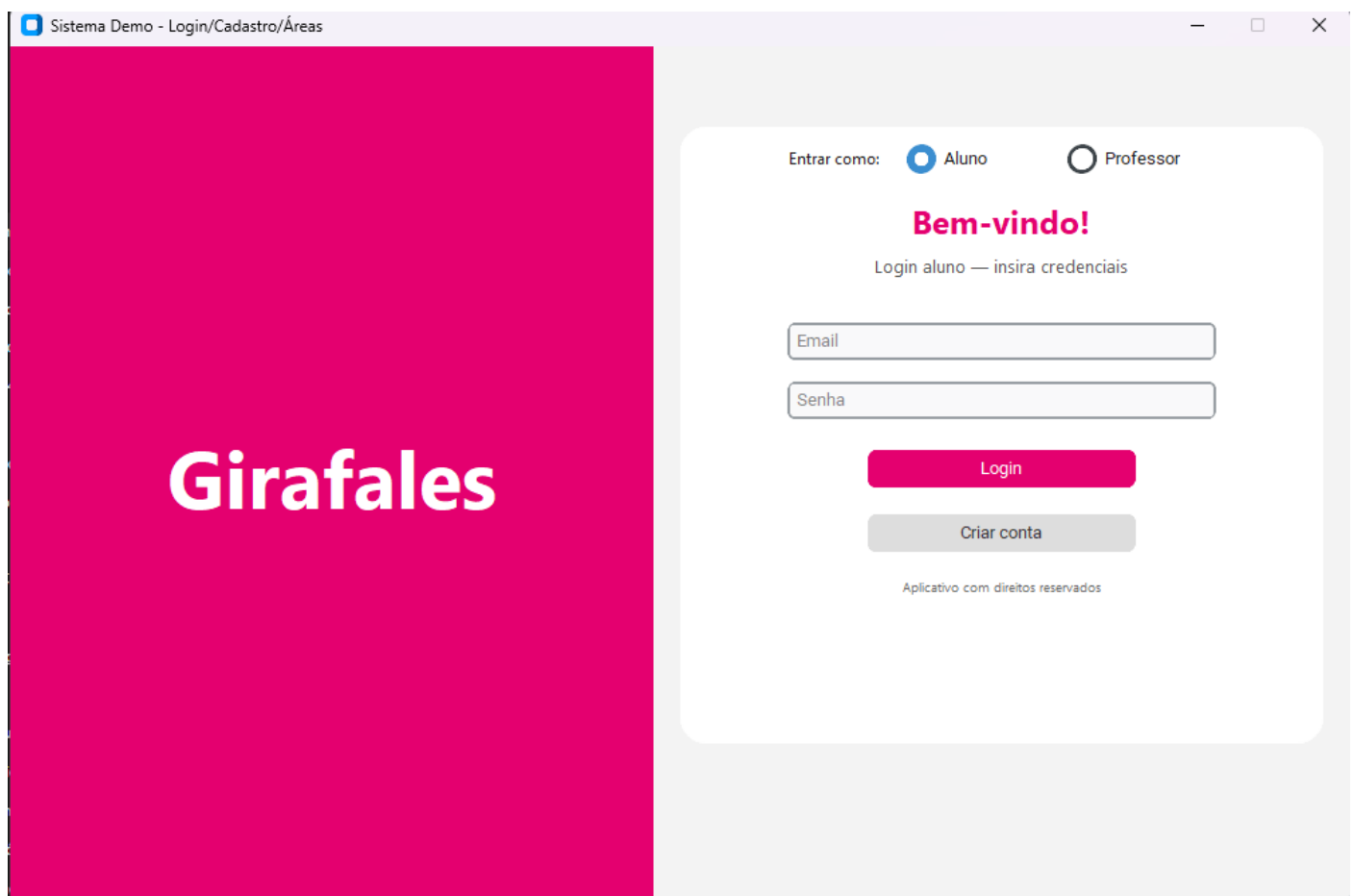


Figura 2 - Login

2. Fluxo de login e autenticação

- Acesso restrito por meio de login funcional, verificando senha e e-mail.
- Diferenciação de interface e funcionalidades entre alunos e professores após autenticação.

3. Área do aluno

- Visualização do boletim escolar, contendo todas as matérias cadastradas e suas respectivas notas.

- Apresentação de informações adicionais, como e-mail do aluno, nome do responsável e idade.



Figura 3 - Area do Aluno

4. Área do professor

- Listagem de todos os alunos cadastrados na escola, com informações detalhadas (nome, e-mail, CPF).
- Lançamento e atualização de notas por disciplina, com validação de entradas.
- Atualização dinâmica da lista de alunos após alterações nas notas.

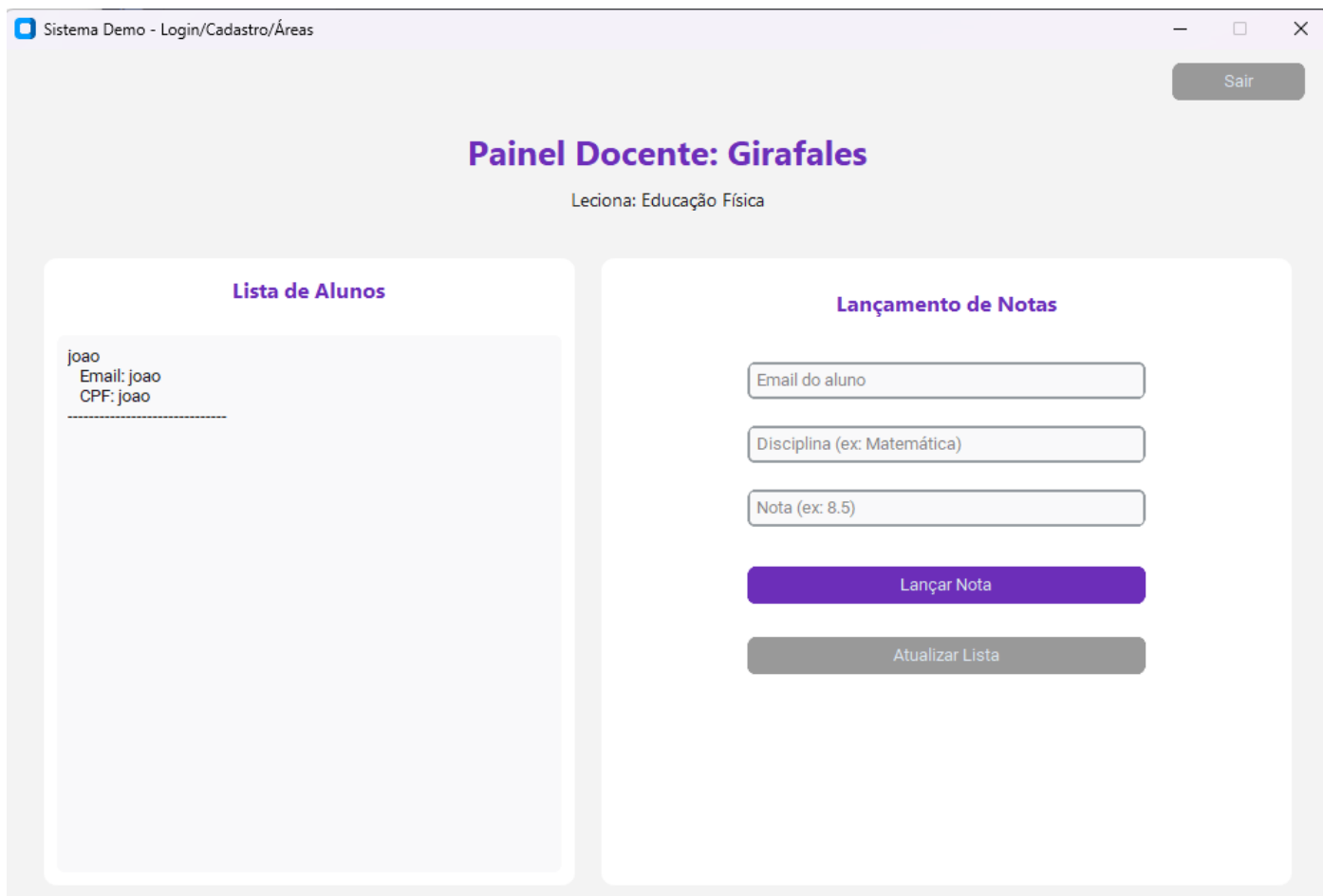


Figura 4 - Area do Professor

5. Estrutura de classes e POO

- Aplicação de conceitos de Orientação a Objetos, incluindo:
 - **Herança:** Usuario como classe base, estendida por Aluno, Professor e Pai.
 - **Encapsulamento:** Atributos sensíveis (como senha) protegidos, com métodos de acesso controlado.
 - **Polimorfismo:** Métodos para manipulação de dados que se comportam de forma distinta dependendo do tipo de usuário.
- Uso de coleções (listas) para armazenar alunos, professores e matérias em memória, permitindo consultas e alterações dinâmicas.

6. Interface gráfica funcional

- Desenvolvimento de GUI utilizando customtkinter integrada com o backend, garantindo interação intuitiva e responsiva.
- Diferenciação visual de cores e elementos para alunos e professores, melhorando a experiência do usuário.

Conclusão

O projeto desenvolvido demonstrou a aplicação prática dos conceitos de Programação Orientada a Objetos em Python, resultando em um sistema funcional para gerenciamento escolar. Através da implementação de classes bem estruturadas, abstração, encapsulamento, herança e polimorfismo, foi possível criar um ambiente que diferencia perfis de usuários e permite operações específicas para cada papel, como cadastro, consulta e lançamento de notas.

A interface gráfica, construída com customtkinter, proporcionou uma experiência interativa e intuitiva, permitindo que alunos e professores acessem e gerenciem informações de forma clara e organizada. A integração entre backend e frontend mostrou-se eficiente, garantindo que dados de usuários e matérias fossem armazenados em memória e manipulados de forma segura.

O sistema atendeu aos requisitos propostos, permitindo o cadastro de múltiplos usuários, consultas por critérios específicos e listagem completa de informações, evidenciando a aplicação de conceitos de POO de maneira prática e didática. Além disso, o projeto possibilitou a compreensão profunda do fluxo de autenticação, gerenciamento de dados e construção de interfaces funcionais, servindo como base sólida para futuros aprimoramentos e extensões.

Como aprendizado, o desenvolvimento deste projeto reforçou a importância do planejamento, da documentação e do uso correto dos princípios de Orientação a Objetos, mostrando que é possível criar sistemas complexos de forma organizada, modular e escalável.